DocScan Project Report

Team Details:-

S. No.	Team Member	Registration No.
1	Satyam Jha	20233254
2	Sanjeevani Jaiswal	20233252
3	Puneet Kumar	20233225
4	Priyanshu Pal	20233223

Abstract

This report covers an extensive analysis of the DocScan project. It is an automatic system that allows digitisation of documents, text extraction and table parsing. The project features a complete full-stack system including modules for image processing, model evaluation and a user interface.

This report will describe how the system was built, how it works and what results were found by looking at the provided artefacts in the repository.

Introduction

Digitising physical documents such as handwritten notes and printed forms is a vital process in data processing. DocScan is a project aimed at enabling this by providing a system for accurately extracting text and tabular data from images. The system has been created using Python. This means that there is an easy to use front-end and powerful back-end.

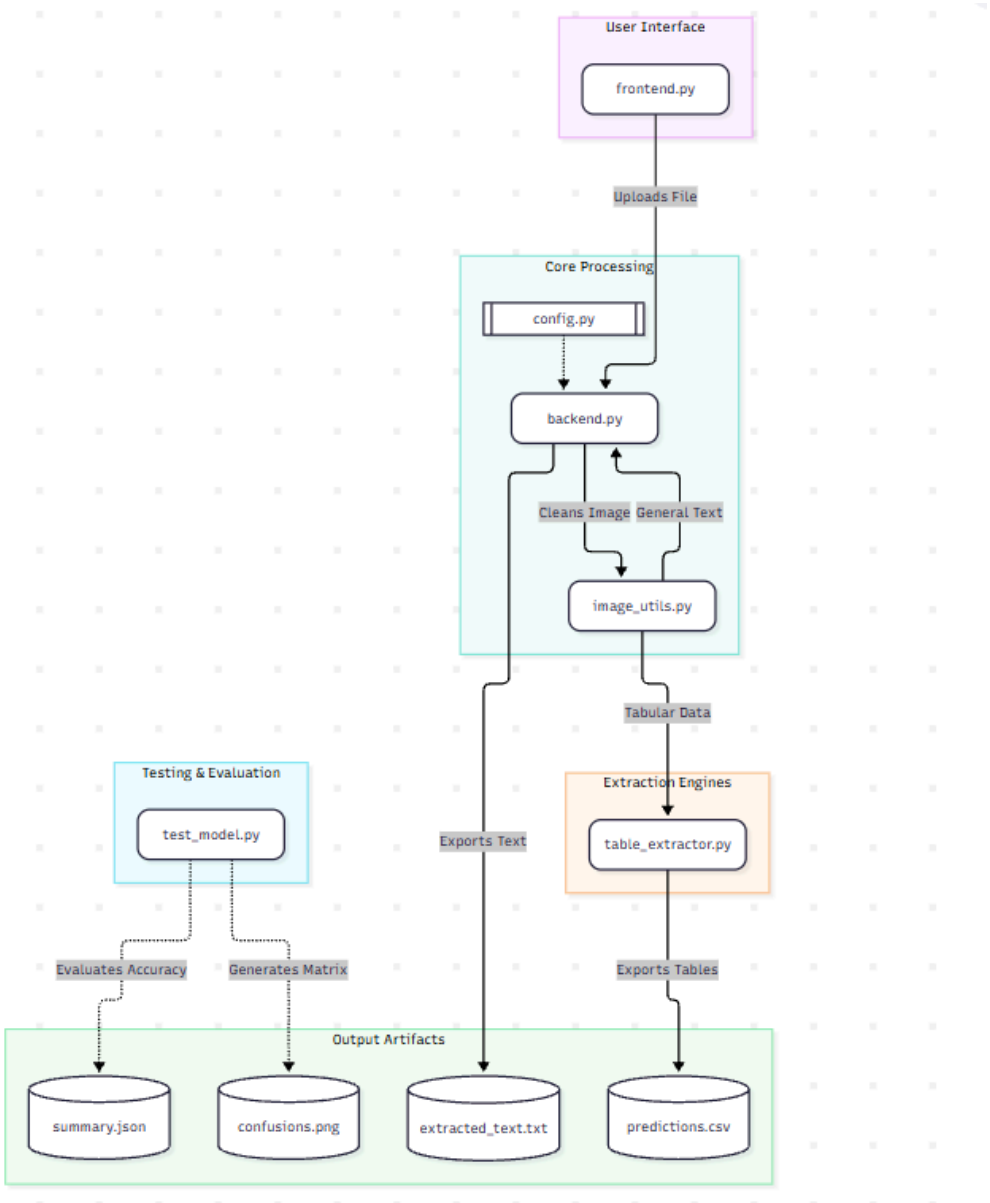
There are many challenges faced when trying to perform OCR on documents. There can be lots of noise, different fonts and complex structures such as tables. DocScan tries to overcome this by having individual scripts for each functionality. This makes the system highly cohesive and loosely coupled.

Methodology

The DocScan system's methodology is divided into several key operational modules:

1. **System Architecture:**

The application is divided into a frontend (frontend.py) for user uploads and interaction, and a backend (backend.py) that orchestrates the processing pipeline. Configuration settings are managed via 'config.py', ensuring environment variables and model paths are easily adjustable.



2. **Image Processing:**

Raw images (both printed and handwritten, as seen in the **test_img** directory) are initially processed using **'image_utils.py'**. This module handles binarization, noise reduction, and deskewing to enhance readability and isolate text regions.

3. Printed Text Recognition

Printed text recognition allows DocScan to extract text from documents such as printed notes, forms, and other typed pages. Since printed characters usually have more consistent shapes and spacing than handwritten text, they can generally be recognized using conventional OCR techniques.

DocScan uses Tesseract OCR as its primary engine for recognizing printed text. It processes the input image, identifies the characters and words, and converts them into editable text. This makes it easier for users to digitize documents without having to type everything manually.

The system also supports EasyOCR as a fallback when the confidence of the initial recognition falls below a predefined threshold. This gives the application another way to process documents when the primary OCR engine may not produce reliable results.

Once the text has been extracted, it is passed back to the backend, which prepares the output for display through the frontend or saves it for further use.

4. Handwriting Recognition

Handwriting recognition is an important part of DocScan because handwritten documents are generally more difficult to process than printed ones. Different writing styles, irregular spacing, and variations in character shapes can make it challenging to identify the correct text.

To handle these challenges, DocScan uses Microsoft's TrOCR model along with several preprocessing and segmentation techniques. Instead of processing the entire document as one image, the system identifies individual lines of handwriting and recognizes them separately.

4.1 Preprocessing and Line Segmentation

Before recognizing the text, the input image is prepared to make the handwriting easier to process. This stage includes correcting the image's polarity, resizing it, correcting skew, and adjusting contrast. The system also attempts to remove

unwanted elements, such as ruled notebook lines and small noise components, that could interfere with recognition.

After preprocessing, the document is divided into individual lines of handwriting. The system uses horizontal projection profiles to examine how foreground pixels are distributed across the image and identify possible boundaries between lines. Smoothing and additional checks help handle cases where lines are close together or the handwriting does not follow a perfectly regular layout.

If a text region is too wide, the system can also look for suitable splitting points and divide it into smaller segments. This helps prepare the input for the recognition model and makes it easier to process long lines of handwriting.

4.2 TrOCR-Based Recognition

DocScan uses Microsoft's TrOCR (Transformer-based Optical Character Recognition) model to recognize handwritten text. Unlike traditional OCR approaches, TrOCR uses a transformer-based architecture to learn visual patterns in an image and generate the corresponding text.

The model consists of two main components: a visual encoder, which extracts useful features from the input image, and a text decoder, which uses those features to generate the predicted text sequence.

The implementation uses a pretrained TrOCR model for handwritten text recognition. To make processing more efficient, individual line images are grouped into batches rather than being processed one at a time. The model also uses beam search during decoding, allowing it to consider multiple candidate text sequences before selecting its output.

4.3 Hybrid Recognition Strategy

Although TrOCR is the primary model for handwritten text, DocScan also uses Tesseract for certain lines identified by an uppercase-handwriting heuristic. These lines are processed using Tesseract's single-line page segmentation mode.

This approach allows the system to use different recognition methods for selected inputs rather than relying entirely on one model. The choice is based on a heuristic in the implementation, so it does not guarantee that Tesseract will produce better results for every uppercase line.

Using this combination of recognition engines provides an alternative way to handle different types of handwritten input within the same application.

4.4 Text Reconstruction and Correction

Once the individual lines have been recognized, their predicted text is combined in the correct order to reconstruct the document's content. This allows the system to produce a continuous text output instead of returning separate predictions for every line.

DocScan also includes dictionary-based spelling correction to help address certain recognition errors. However, spelling correction is not always reliable, particularly when dealing with names, uncommon words, or technical terms. For this reason, the quality of the final output depends on both the recognition model and the effectiveness of the correction process.

After processing is complete, the reconstructed text is returned to the backend for output generation and displayed through the frontend.

5. Table Structure Recognition and Extraction

Extracting information from tables requires more than simply recognizing the text within them. Unlike ordinary paragraphs, tables contain structured information arranged in rows, columns, and individual cells. Therefore, the system must identify both the text and its position to ensure that the extracted information retains the same meaning and organization as the original table.

In DocScan, table extraction is handled by the dedicated `table_extractor.py` module. The process begins by loading the table image and applying image preprocessing techniques to improve its quality and make the table structure easier to detect. These steps may include converting the image to grayscale, applying thresholding to separate text and lines from the background, and resizing the image when necessary.

After preprocessing, the system attempts to identify the table boundaries and cell structure. For tables with visible borders, image processing techniques are used to detect horizontal and vertical lines. These lines help determine the boundaries of individual cells. The system then identifies the enclosed regions and estimates their row and column positions. When table borders are missing or unclear, detecting the cell structure becomes more difficult, and the system must rely on the positions and shapes of the detected regions.

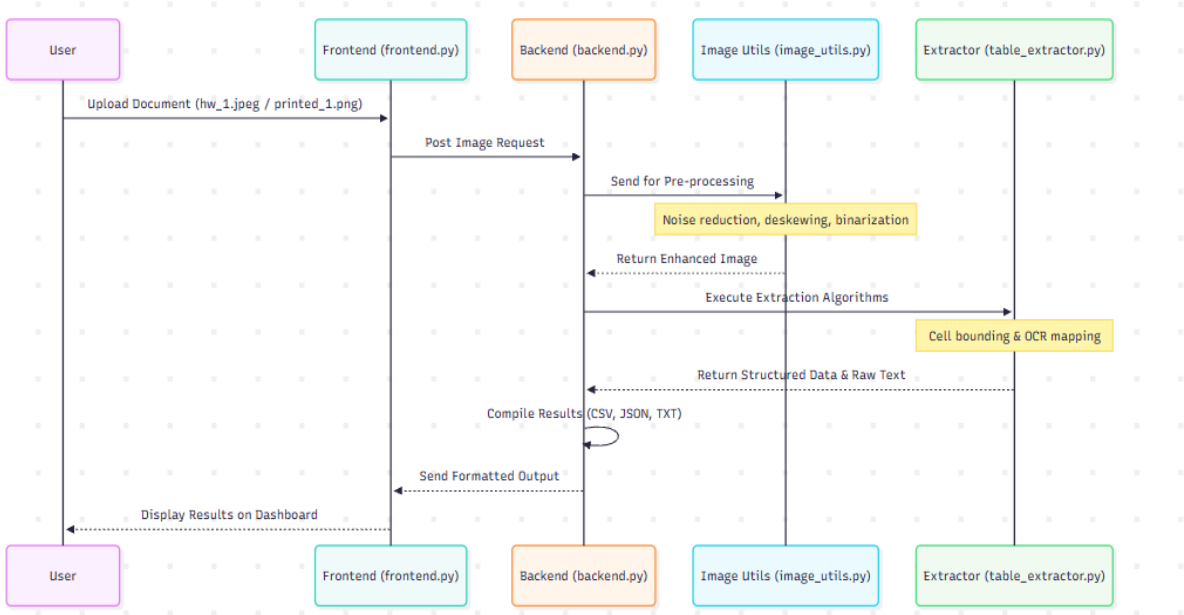
Once the cells have been identified, Optical Character Recognition (OCR) is used to extract the text from each cell. The recognized text is associated with its corresponding row and column, allowing the system to reconstruct the table in a structured format. This step is important because correctly recognizing the text alone is not sufficient; each value must also remain associated with the appropriate heading, row, and column.

Finally, the extracted information can be organized into a grid and saved in a structured output format, such as CSV, for further use. This makes the data easier to review, edit, analyze, and process using other applications. The approach is particularly useful for extracting information from academic documents, reports, business records, and other documents containing tabular data.

However, the accuracy of table extraction depends on the quality of the input image and the clarity of its structure. Skewed images, low resolution, broken borders, closely spaced text, and tables without clear cell boundaries can affect detection. These issues may result in missing text, incorrectly identified cells, or values being assigned to the wrong rows and columns. Therefore, image preprocessing and accurate cell detection are essential for producing reliable, structured table data.

6. **Model Testing and Validation:**

The underlying extraction model is evaluated using 'test_model.py', which tests the system against a variety of inputs (e.g., hw_1.jpeg, printed_2.jpg, table_4.png) to validate accuracy across different document types.



Results and Discussion

The repository's 'results' directory highlights the effectiveness of the extraction pipeline. The system outputs structured predictions into 'predictions.csv' and aggregates metadata into 'summary.json'.

Visual analysis of the model's accuracy is provided via 'confusions.png', which acts as a confusion matrix. This matrix helps identify misclassifications between different character or structural classes (e.g., confusing 'O' with '0', or merging adjacent table cells).

Testing was conducted on a diverse dataset containing:

- Handwritten documents (e.g., hw_1.jpeg to hw_5.jpg)
- Printed documents (printed_1.png to printed_5.png)
- Complex tables (table_3.jpg to table_img_2.jpg)

The inclusion of a dedicated table extractor successfully parses structured images, demonstrating the system's versatility and robustness across varying degrees of document complexity.

Conclusion

The DocScan project provides a highly functional, modular approach to document scanning and text extraction. By combining general optical character recognition techniques with specialized table extraction algorithms, it successfully bridges the gap between unstructured physical images and structured digital data.

Future iterations of the DocScan project can leverage the insights from the confusion matrix to further fine-tune the model. Additionally, expanding the 'table_extractor.py' to handle borderless tables or highly distorted scans would further enhance the project's utility in real-world scenarios.